Retrieval-Augmented Generation (RAG)

From Production Deployment to Privacy-Preserving Multilingual Systems

13 TOPICS Covered

Ch 1 — Deploying and Maintaining RAG Systems
Ch 2 — Evaluating RAG Solutions
Ch 3 — Optimizing Data Retrieval Techniques
Ch 4 — Modular RAG Architectures
Ch 5 — Improving Retrieval with RAG Fine-Tuning
Ch 6 — Vector Space Models and Embeddings in RAG
Ch 7 — Expansion Strategies for RAG
Ch 8 — Integrating Knowledge Bases for RAG Systems
Ch 9 — Relevance and Scoring Mechanisms for RAG
Ch 10 — Multi-modal RAG Systems
Ch 11 — Fact Verification with RAG Systems
Ch 12 — Federated Learning and Privacy-Preserving RAG
Ch 13 — Multilingual Generative AI: Cross-Lingual RAG Systems

2025 Edition • Intermediate Level

Chapter 1: Deploying and Maintaining RAG Systems

Section I — Theory & Conceptual Foundations

1.1 Introduction

Building a Retrieval-Augmented Generation (RAG) prototype is relatively straightforward. Deploying and maintaining a production-grade RAG system is where true engineering begins. A production RAG platform must be reliable, scalable, cost-efficient, secure, observable, continuously updated, low-latency, and easy to maintain.

Many teams successfully build demos but fail when real users, enterprise data, security constraints, and operational complexity arrive. This chapter explains how to deploy and sustain enterprise RAG systems professionally.

1.2 Why Production RAG Is Different from Demo RAG

Demo RAG	Production RAG
PDF → Embeddings → Vector DB → LLM	Multi-source ingestion pipeline
Good for proof of concept	Versioned indexing + access control
No operational concerns	Monitoring, logging, retry systems
Single developer	High availability + disaster recovery
Static data	Continuous refresh pipelines + governance

1.3 Core Production Architecture

A well-designed RAG system has two primary pipelines:

Pipeline A — Ingestion Pipeline (Offline)

Responsible for preparing and indexing knowledge. Runs as a batch process, separate from the serving path.

1. Collect source data from all enterprise knowledge sources
2. Parse documents (PDF, Word, HTML, SQL, APIs)
3. Clean content — remove noise, fix encoding, deduplicate
4. Chunk text into appropriately-sized retrieval units
5. Generate embeddings for each chunk
6. Store embeddings and metadata in the vector database
7. Build metadata indexes to support filtered search

Pipeline B — Query Pipeline (Online)

Responsible for answering user queries in real time.

8. Receive user query
9. Authenticate and authorize the user
10. Embed the query using the same embedding model as ingestion
11. Retrieve the top relevant chunks via vector similarity search
12. Re-rank results using a cross-encoder or learned ranker
13. Construct the augmented prompt combining query + retrieved context
14. Call the LLM to generate a grounded response
15. Return the response with optional citations

1.4 Deployment Components

Layer	Purpose & Examples
Frontend Layer	Chat UI, internal employee portal, customer support portal — the interface users interact with
API Layer	FastAPI, Flask, or Node.js — the bridge between the frontend and retrieval + generation logic
Retriever Layer	Embeds queries, executes vector search, and applies metadata filters to narrow results
Vector Database Layer	Pinecone, Chroma, Weaviate, Milvus, or Elasticsearch — stores and serves embedding vectors
LLM Layer	OpenAI GPT models, Anthropic Claude, Gemini, or self-hosted open models
Monitoring Layer	Tracks latency, failures, retrieval relevance, cost per query, and user satisfaction signals

1.5 Infrastructure Deployment Models

Model	Best For	Key Tools
Cloud Managed	Fast-moving teams; managed scaling	AWS, Azure, GCP — managed DBs, security tools
Self-Hosted	Sensitive enterprises needing data sovereignty	Docker, Kubernetes, private VPC
Hybrid	Common enterprise pattern — private retrieval + external LLM	Private vector DB + OpenAI/Claude API

1.6 Containerization and Kubernetes Strategy

Use Docker to package each service independently. Kubernetes provides auto-scaling, rolling updates, load balancing, failover, and resource quotas at scale.

- Containerize each component separately: API service, ingestion worker, retriever service, UI frontend

- Benefits: portability, environment consistency, and easier CI/CD pipelines
- Kubernetes use case: auto-scale retrievers during peak load; rollback deployments safely

1.7 Data Refresh Strategy

Knowledge changes constantly. Without scheduled refresh pipelines, your RAG system answers from stale data. Typical schedules:

- Every hour: product catalog, live pricing, inventory
- Daily: policy documents, HR manuals, knowledge base articles
- Weekly or on-change: legal archives, compliance regulations

1.8 Common Maintenance Challenges

Challenge	Root Cause & Impact
Stale Knowledge	Old embeddings from outdated documents — system confidently answers with wrong information
Broken Connectors	Source system API changed but ingestion pipeline was not updated — silent failure
Cost Spikes	Traffic increased unexpectedly — embedding + LLM API costs escalate rapidly
Slow Search	Large index growth without ANN optimization — retrieval latency degrades user experience
Hallucination Drift	Prompt changes interact badly with retrieval results — answer quality quietly degrades

1.9 Architect Perspective

Deploying RAG Is Multi-Disciplinary Engineering

Search engineering — retrieval quality, indexing strategy, ANN optimization
Data engineering — ingestion pipelines, refresh schedules, metadata design
Backend engineering — API design, authentication, caching, rate limiting
MLOps — model versioning, evaluation pipelines, A/B testing
Security engineering — access control, PII handling, secrets management

Section II — Practical & Coding

In this section, we build a complete deployable RAG backend, containerize it with Docker, and add a scheduled re-indexing job — covering the full production deployment lifecycle.

Part A — FastAPI RAG Service

Cell 1 — Install Packages

```
pip install fastapi uvicorn langchain chromadb openai
```

This installs FastAPI (the web framework), Uvicorn (the ASGI server), LangChain (the orchestration layer), ChromaDB (a local vector store), and the OpenAI SDK.

Cell 2 — Imports

```
from fastapi import FastAPI
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI
```

`FastAPI` creates the web application. `Chroma` is our vector store. `OpenAIEmbeddings` converts text to dense vectors. `RetrievalQA` is a pre-built LangChain chain that wires retrieval + generation together.

Cell 3 — Create App

```
app = FastAPI()
```

Creates the FastAPI application instance. Every route we define will be registered on this object.

Cell 4 — Load Vector Database

```
vectordb = Chroma(
    persist_directory="db",
    embedding_function=OpenAIEmbeddings()
)
```

`persist_directory="db"` tells Chroma to load from (or create) a local folder called 'db'. `embedding_function=OpenAIEmbeddings()` means every query will be embedded using OpenAI's text-embedding model before vector search is performed.

Cell 5 — Build QA Chain

```
qa = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    retriever=vectordb.as_retriever()
)
```

`RetrievalQA.from_chain_type()` creates a complete question-answering pipeline: retrieve relevant chunks from the vector DB, inject them into a prompt template, and call the LLM. `as_retriever()` wraps the vector store in a standard retriever interface.

Cell 6 — Create API Endpoint

```
@app.get("/ask")
def ask(query: str):
    return {"answer": qa.run(query)}
```

`@app.get("/ask")` registers a GET endpoint at the URL /ask. The `query: str` parameter is automatically parsed from the URL query string (e.g., /ask?query=refund+policy). The chain runs and returns a JSON object with the answer.

Cell 7 — Run Server

```
uvicorn main:app --reload
```

`uvicorn main:app` tells Uvicorn to look for an object called `app` inside `main.py`. `--reload` enables hot-reloading during development. Remove `--reload` in production — use `--host 0.0.0.0 --port 8000` instead for container deployment.

Part B — Docker Deployment

Cell 8 — Dockerfile

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- `FROM python:3.11` — starts from the official Python 3.11 Docker image, giving us a clean Python environment.
- `WORKDIR /app` — sets the working directory inside the container to /app. All subsequent commands run here.
- `COPY . .` — copies everything in your local directory (including main.py, requirements.txt, and the db/ folder) into the container.
- `RUN pip install -r requirements.txt` — installs all dependencies at image build time, so they are baked into the image.
- `CMD [...]` — the command that runs when a container starts. Binding to 0.0.0.0 (all interfaces) is required for Docker port forwarding to work.

```
# Build the Docker image
docker build -t rag-app .

# Run the container, mapping port 8000
docker run -p 8000:8000 rag-app
```

Part C — Scheduled Reindex Job

Cells 9–11 — Automated Knowledge Refresh

```
import schedule
import time

def refresh_embeddings():
    """Re-ingests all source documents and rebuilds the vector index."""
    print("Reindexing documents...")
    # Add your actual ingestion logic here:
    # 1. Load documents from source
    # 2. Chunk and embed
```

```
    # 3. Overwrite the vector store

# Schedule the refresh to run every day at 02:00 AM
schedule.every().day.at("02:00").do(refresh_embeddings)

# Keep the scheduler running
while True:
    schedule.run_pending()
    time.sleep(60)  # Check every minute
```

- `schedule.every().day.at("02:00")` — defines a schedule that fires the `refresh_embeddings` function once per day at 2 AM. The `schedule` library handles the timing, so you do not need a cron job.
- Running this as a separate Docker container alongside the API service is a clean production pattern — the ingestion worker runs independently without affecting the serving path.

Chapter 2: Evaluating RAG Solutions

Section I — Theory & Conceptual Foundations

2.1 Why Evaluation Matters

A RAG system may look impressive in demos yet fail badly in production. Common failures include: retrieving wrong documents, giving incomplete answers, hallucinating unsupported facts, slow responses that frustrate users, and inconsistent quality across domains. Because RAG combines retrieval and generation, evaluation must test both systems independently. Without measurement, teams optimize blindly.

2.2 The Two Core Layers to Evaluate

Layer	Key Evaluation Questions
Layer 1: Retrieval System	Did it retrieve the right chunks? Were the best documents ranked highly? Was search fast enough?
Layer 2: Generation System	Is the answer factually grounded in the retrieved context? Did it address user intent? Is it complete and concise?

2.3 Types of Evaluation

Evaluation Type	Description & When to Use
Offline Evaluation	Run benchmark datasets during development to measure baseline metrics before deployment
Online Evaluation	Measure real user traffic in production — the ground truth for actual system quality
Human Evaluation	Expert reviewers judge outputs manually — expensive but highest quality signal
Automated Evaluation	Use metrics, LLM-as-judge, and scripts for scalable quality measurement

Best Practice

Strong evaluation programs combine all four types.
Offline evaluation catches regressions before deployment.
Online evaluation reveals real-world failure modes that benchmarks miss.
Human evaluation is the gold standard for validating automated metrics.

2.4 The Golden Dataset Concept

Create a benchmark dataset — a regression suite of curated query/expected-answer pairs — that you run against your system on every release.

Query	Expected Source	Expected Answer
Refund policy for damaged laptops	Returns Policy PDF, page 4	60-day refund window
Can leave balance carry forward?	HR Handbook, Section 3	Carry-forward is allowed annually
Invoice approval threshold?	Finance SOP v2	Amounts above ₹50,000 require manager approval

This golden dataset becomes your system's regression suite — any degradation in benchmark scores triggers investigation before it reaches users.

2.5 Common Failure Categories

Failure Type	Description
Retrieval Failure	The correct answer exists in the corpus but the wrong documents were retrieved — a retrieval quality problem
Ranking Failure	The right document was retrieved but ranked too low to enter the LLM's context window
Chunking Failure	Relevant information was split across chunk boundaries, preventing retrieval of complete evidence
Prompt Failure	Good documents were retrieved but a poorly constructed prompt caused a weak answer
Hallucination Failure	The generated answer is not supported by any retrieved evidence — the model invented facts
UX Failure	The answer is technically correct but too slow or confusingly formatted for the user to act on

2.6 Key Retrieval Metrics

Metric	Definition
Precision@K	Among the top K retrieved documents, what fraction are actually relevant to the query?
Recall@K	Of all the relevant documents that exist, what fraction did the system retrieve in the top K?
MRR (Mean Reciprocal Rank)	Measures how early the first relevant document appears. MRR = (1/N) × Σ (1/rank_i)
NDCG	Normalized Discounted Cumulative Gain — measures ranking quality with graded relevance scores

2.7 Key Generation Metrics

Metric	What It Measures

Faithfulness	Is every claim in the answer supported by the retrieved source documents?
Relevance	Does the answer actually address what the user was asking?
Correctness	Is the answer factually accurate compared to ground truth?
Completeness	Does the answer include all the information needed to fully answer the question?
Conciseness	Is the answer free of unnecessary verbosity and repetition?

2.8 Continuous Improvement Loop

RAG quality changes over time as documents are updated, prompts evolve, and user queries drift. Evaluation must be ongoing — not a one-time activity.

```
Query Logs
  ↓
Failure Detection (automated metric drops)
  ↓
Root Cause Analysis (retrieval? chunking? prompt?)
  ↓
Fix the Identified Layer
  ↓
Re-test Against Golden Benchmark
  ↓
Deploy with Confidence
```

Section II — Practical & Coding

Part A — Sample Test Dataset (Cell 1)

```
test_cases = [
    {
        "query":            "What is refund period for damaged laptops?",
        "expected_keyword": "60 days"
    },
    {
        "query":            "Can leave balance carry forward?",
        "expected_keyword": "carry"
    }
]
```

Each test case pairs a realistic user query with a keyword that must appear in a correct answer. This is a lightweight but practical evaluation harness — the expected_keyword acts as a signal of whether the correct information reached the LLM and appeared in the response.

Part B — Simulated Answers (Cell 2)

```
predictions = [
    "Damaged laptops can be returned within 60 days.",
```

```
    "Unused leave can carry forward annually."
]
```

In a live system, these predictions would come from your RAG pipeline. Here they are hardcoded to demonstrate the evaluation logic in isolation.

Part C — Basic Accuracy Check (Cell 3)

```
score = 0
for i in range(len(test_cases)):
    keyword = test_cases[i]["expected_keyword"]
    if keyword.lower() in predictions[i].lower():
        score += 1

accuracy = score / len(test_cases)
print(f"Accuracy: {accuracy:.1%}")
```

`keyword.lower() in predictions[i].lower()` — performs a case-insensitive substring check. If the expected keyword appears in the prediction, that test case passes. This is a simple but fast automated relevance proxy, useful as a sanity check before deploying more expensive human evaluation.

Part D — Retrieval Recall@K (Cells 4–6)

```
# Simulated top-2 retrieved documents per query
retrieved_docs = [
    ["refund policy page", "shipping page"],
    ["holiday rules",      "leave handbook"]
]

# The single correct document that should have been retrieved
ground_truth = [
    "refund policy page",
    "leave handbook"
]

# Calculate Recall@2
hits = 0
for i in range(len(ground_truth)):
    if ground_truth[i] in retrieved_docs[i]:
        hits += 1

recall_at_2 = hits / len(ground_truth)
print(f"Recall@2: {recall_at_2:.1%}")
```

Recall@K answers the question: 'Was the correct source document present anywhere in the top K results?' This is the most important retrieval metric for RAG — if the correct document is not retrieved at all, no amount of LLM quality can compensate.

Part E — Latency Logging (Cells 7–8)

```
import time

# Wrap any RAG call with timing measurement
```

```
start = time.time()

# --- Your RAG pipeline call goes here ---
# result = qa.run(query)

latency_ms = (time.time() - start) * 1000
print(f"Latency: {latency_ms:.0f} ms")
```

Why Latency Is a First-Class Metric

A correct answer delivered in 8 seconds is often useless in real-time user interfaces.
Latency directly impacts user adoption and satisfaction.
Track p50, p95, and p99 latencies — not just the average.
Retrieval latency and LLM generation latency should be measured separately to identify bottlenecks.

Chapter 3: Optimizing Data Retrieval Techniques

Section I — Theory & Conceptual Foundations

3.1 Why Retrieval Optimization Matters

In RAG, many teams focus heavily on the language model. But in practice: a weak retriever can make even the best LLM fail. If irrelevant or incomplete context is retrieved, the model cannot produce reliable answers. Elite RAG systems treat retrieval as a first-class engineering discipline.

3.2 Symptoms of Poor Retrieval

- Wrong answers despite correct documents existing in the corpus
- Irrelevant chunks appearing in top results, consuming the LLM's context window
- Missing critical clauses or facts because relevant information was split across chunks
- Slow search performance degrading user experience
- High token cost due to noisy, non-relevant context being included in prompts
- Hallucinations caused by weak grounding — the model fills in what retrieval missed

3.3 Types of Retrieval Systems

Type	Mechanism	Best For
Keyword Retrieval (BM25/TF-IDF)	Exact term matching with frequency weighting	IDs, exact names, policy numbers, legal clause references
Dense Retrieval (Vector Search)	Embedding similarity in high-dimensional space	Natural language queries, meaning-based search, synonyms
Hybrid Retrieval	Combines keyword + semantic scores	Production systems — usually the strongest practical choice

Why Vector Search Alone Can Fail

Query: 'What is policy 17B for returns?'
Pure semantic retrieval may miss the exact identifier '17B' because embeddings focus on meaning, not exact tokens.
Keyword retrieval catches it because BM25 scores exact term matches.
Hybrid systems get both: semantic understanding AND exact-match precision.

3.4 Query Understanding Layer

Before retrieval, interpret user intent. A user asking 'refund damaged laptop after 45 days' contains multiple signals: the topic (refund policy), the product category (laptop), the condition (damaged), and a time threshold (45 days). Detecting these signals individually improves retrieval precision by enabling targeted metadata filters alongside the embedding search.

3.5 Metadata Filtering

Use document metadata to reduce the search space before vector similarity comparison. This speeds search and dramatically improves precision without requiring better embeddings.

```
# Example metadata filter
{
    "department": "HR",
    "year": 2026,
    "country": "India",
    "doc_type": "policy"
}
```

3.6 Re-ranking Layer

The initial retriever is optimized for speed — it retrieves 20–100 candidates quickly. A second, slower, but more accurate model (a cross-encoder) then scores each candidate against the query in a pairwise fashion. This re-ranking step often provides major quality improvements by surfacing the most relevant chunk to the top of the list.

3.7 Chunking Optimization

Chunking Problem	Impact on Retrieval
Chunks too large	Each chunk contains mixed topics — embeddings become blurred and imprecise
Chunks too small	Critical context is split; neither chunk alone answers the question
No overlap between chunks	Sentences near chunk boundaries may be irretrievable even if semantically relevant
Same chunk size for all content	Legal clauses, tables, and narrative prose have fundamentally different optimal sizes

Best practice: use semantic or recursive chunking strategies with a chunk size around 800 tokens and a 100-token overlap at boundaries. Overlap ensures that important sentences crossing boundaries remain searchable from either chunk.

3.8 Top-K Optimization

How many chunks to retrieve (K) is a critical tuning parameter:

- K too low: miss critical evidence, leading to incomplete or wrong answers
- K too high: adds noise to the prompt, increases token cost, and can confuse the LLM
- Typical experiments: K = 3, 5, 8, 10 — measure best recall/precision trade-off on your golden dataset

3.9 Production Retrieval Architecture

```
User Query
  ↓
Query Rewriter (improve intent clarity)
```

```
  ↓
Parallel Search:
  ├── BM25 (keyword match)
  ├── Vector Search (semantic match)
  └── Metadata Filter (narrow scope)
  ↓
Merge and Deduplicate Results
  ↓
Re-ranker (cross-encoder precision scoring)
  ↓
Top Context (K best chunks)
  ↓
LLM (grounded response generation)
```

Section II — Practical & Coding

Part A — BM25 Keyword Retrieval (Cells 1–4)

```
from rank_bm25 import BM25Okapi

# Corpus of documents
docs = [
    "Policy 17B covers damaged laptop returns.",
    "Shipping timelines for electronics.",
    "Employee leave carry forward rules."
]

# Tokenize the corpus
tokenized = [doc.lower().split() for doc in docs]
bm25 = BM25Okapi(tokenized)

# Score each document against the query
query = "policy 17B returns"
scores = bm25.get_scores(query.lower().split())
print("BM25 scores:", scores)
```

`BM25Okapi(tokenized)` builds the BM25 index from the tokenized corpus. `bm25.get_scores(query.lower().split())` returns a score for each document based on term frequency × inverse document frequency × length normalization. The first document scores highest because it contains all three query terms: 'policy', '17B', and 'returns'.

Part B — Vector Retrieval (Cells 5–7)

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

# Create a vector store from the corpus
vectordb = Chroma.from_texts(docs, OpenAIEmbeddings())

# Semantic search — note the query uses different words than the corpus
results = vectordb.similarity_search(
    "How can I return a damaged notebook computer?",
    k=2
)
```

The query uses 'notebook computer' and 'return', while the document says 'laptop' and 'returns'. Vector search handles this because embeddings capture meaning, not just exact words. The embedding model learns that 'laptop' and 'notebook computer' refer to the same device, so their vectors are close in the embedding space.

Part C — Hybrid Merge and Re-ranking (Cells 8–11)

```
# Simulate keyword and vector results
keyword_results = ["Policy 17B covers damaged laptop returns."]
vector_results  = [
    "Policy 17B covers damaged laptop returns.",
    "Shipping timelines for electronics."
]

# Merge and deduplicate (union of both result sets)
merged = list(set(keyword_results + vector_results))

# Simple word-overlap re-ranker
def rerank(query, docs):
    q = set(query.lower().split())
    return sorted(docs, key=lambda d: len(q & set(d.lower().split())),
reverse=True)

query  = "policy 17B returns"
ranked = rerank(query, merged)
print(ranked)
```

`set(keyword_results + vector_results)` takes the union of both result sets, ensuring that a document appearing in both is not counted twice. The `rerank` function scores documents by counting how many query words appear in the document — a simple but effective heuristic for demonstration. In production, replace this with a cross-encoder model (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`) for much stronger ranking.

Chapter 4: Modular RAG Architectures

Section I — Theory & Conceptual Foundations

4.1 Introduction: Why Monolithic RAG Fails at Scale

Early RAG systems were often built as a single tightly coupled pipeline: Documents → Embeddings → Vector Search → Prompt → LLM Answer. This works for prototypes but becomes difficult to scale, maintain, and evolve. Modular RAG Architectures separate the system into independent components that can be improved or replaced without rebuilding everything.

4.2 Problems with Monolithic RAG Design

Problem	Impact
Tight Coupling	Changing the embedding model may break retrieval logic; components are not independently testable
Slow Innovation	Cannot swap one component (e.g., the re-ranker) without risking breakage in the rest of the pipeline
Poor Debugging	When quality degrades, it is hard to isolate whether the fault lies in chunking, retrieval, or generation
Scaling Problems	Search and generation scale differently — monolithic systems cannot optimize each independently
Vendor Lock-In	Entire stack tied to one provider; switching models requires rewriting the pipeline

4.3 Benefits of Modular Design

Benefit	Description
Flexibility	Swap the vector DB without changing the prompt builder or API layer
Faster Experimentation	A/B test multiple retrievers or chunkers without touching other layers
Better Reliability	Failures are isolated to one module; the rest of the system continues operating
Team Parallelism	Different teams own different modules — retrieval team and generation team work independently
Cost Optimization	Use premium components (expensive re-rankers) only where they provide measurable ROI

4.4 Typical Modules in a Production Modular RAG System

Module	Responsibility	Examples
Data Connectors	Connect to enterprise knowledge sources	SharePoint, Google Drive, Salesforce, SQL databases, wikis
Processing Pipeline	Clean and normalize ingested content	OCR, cleaning, deduplication, metadata extraction
Chunking Engine	Split documents into retrieval-ready units	Different chunkers for legal docs, tables, code, chat logs
Embedding Layer	Generate dense vector representations	OpenAI, Sentence-Transformers, domain-specific models
Retrieval Layer	Execute search against the vector store	Keyword / vector / hybrid retrieval strategies
Ranking Layer	Re-score retrieved candidates precisely	Cross-encoder re-rankers
Generation Layer	Produce the final response	LLM with augmented prompt
Observability Layer	Track quality, cost, and failures	LangSmith, Prometheus, custom dashboards

4.5 Evolution Path from Prototype to Production

```
Phase 1: Single notebook prototype
         (all logic in one file)

Phase 2: API + Vector DB
         (FastAPI endpoint + persistent vector store)

Phase 3: Separate ingestion and serving
         (ingestion worker runs independently on schedule)

Phase 4: Independent modules with CI/CD
         (each module has its own tests, deployment, and versioning)
```

4.6 Architect Perspective

Modular RAG Is the Microservices Mindset Applied to GenAI

Each module has a well-defined input/output interface — like a microservice contract.
Modules can be versioned, tested, and deployed independently.
Finance teams can use a strong ranking model; HR teams can use different chunking — same infrastructure.
Future-proof systems depend on replaceable components.

Section II — Practical & Coding

Part A — Define Modules (Cells 1–5)

```
class DocumentLoader:
    def load(self):
        return [
            "Refund policy for laptops is 60 days.",
```

```
            "Leave balance may carry forward."
        ]

class Chunker:
    def split(self, docs):
        return docs  # Simple pass-through for this demo

class Embedder:
    def embed(self, text):
        return [len(text)]  # Mock embedding — replace with real model

class Retriever:
    def search(self, query, docs):
        return [d for d in docs if any(w in d.lower() for w in query.split())]

class Generator:
    def answer(self, query, context):
        return f"Based on retrieved docs: {context[0]}"
```

Each class represents one independent module. The key design principle: no module knows about the internals of any other module. They communicate only through their defined interfaces (load, split, embed, search, answer). This is exactly the contract-based design that makes modules independently swappable.

Part B — Orchestrator (Cells 6–7)

```
# Instantiate modules
loader    = DocumentLoader()
chunker   = Chunker()
retriever = Retriever()
generator = Generator()

# Execute the pipeline
docs    = loader.load()
chunks  = chunker.split(docs)
results = retriever.search("refund policy", chunks)
print(generator.answer("refund policy", results))
```

The orchestrator wires the modules together without knowing their implementations. If you replace Retriever with BetterRetriever, the orchestrator code does not change. This is the core value of modular design.

Part C — Swap Retriever Without Touching Rest (Cells 8–9)

```
# Original retriever: keyword-based search
class Retriever:
    def search(self, query, docs):
        return [d for d in docs if query.lower() in d.lower()]

# Better retriever: return top-1 result regardless of approach
class BetterRetriever:
    def search(self, query, docs):
        return docs[:1]  # Could be semantic search, re-ranker, etc.

# Only one line changes in the orchestrator:
retriever = BetterRetriever()
```

```
# Everything else remains identical — same orchestrator, same generator
```

Key Lesson — Modular Design in Action

Only one layer changed. The orchestrator, generator, and chunker remained identical.
This is how modular RAG enables rapid A/B testing of individual components.
In production: you can run BetterRetriever on 10% of traffic while the original handles 90%.

Chapter 5: Improving Retrieval with RAG Fine-Tuning

Section I — Theory & Conceptual Foundations

5.1 When Generic Retrieval Fails

Off-the-shelf embedding models are trained on general internet text. In specialized domains — finance, legal, healthcare, insurance, manufacturing — the vocabulary, acronyms, and conceptual relationships differ significantly from general language. A query like 'explain basis risk in swaps' may confuse a generic embedding model, while a finance-tuned model understands the relationship between basis risk, hedging, derivatives, and interest rate risk.

5.2 What Fine-Tuning in RAG Actually Means

Fine-tuning in RAG does not typically mean retraining the full LLM. It means improving one or more retrieval components:

Component to Fine-Tune	What Improves
Embedding Model	Better semantic understanding of domain vocabulary — finance, legal, medical terms
Dual Encoder Retriever	Separate query encoder and document encoder trained together on relevance pairs
Cross-Encoder Re-ranker	Strong pairwise relevance scoring — most expensive but highest quality
Query Rewriter	Learns to translate vague user queries into better search intent
Generator Model	Adapts output style, formatting, and domain reasoning for your use case

5.3 When Fine-Tuning Is Worth It

Fine-Tune When	Do NOT Fine-Tune When
High-value repeated queries (daily enterprise search)	Small internal bot with light usage
Specialized vocabulary (medical/legal/finance)	Insufficient labeled training data (< 500 pairs)
Generic retrieval benchmarks are measurably weak	Knowledge changes rapidly — RAG indexing is the real solution
Large-scale use where ROI justifies investment	Prompting and chunking improvements not yet exhausted

Architect Rule

> Fine-tune only when there is clear, measured retrieval pain — not based on hype.
> First exhaust: better chunking → hybrid retrieval → metadata filters → query expansion → prompt design.
> Then fine-tune only if benchmark metrics are still insufficient.

5.4 Training Data Format for Retrieval Fine-Tuning

Fine-tuning a retriever requires labeled relevance triplets: (query, positive_doc, negative_doc). These triplets teach the model to embed the query close to the positive document and far from the negative one in vector space.

Section II — Practical & Coding

Part A — Prepare Training Triplets (Cell 1)

```
training_pairs = [
    {
        "query":    "refund damaged laptop",
        "positive": "Damaged laptops may be returned within 60 days.",
        "negative": "Shipping timelines for electronics."
    },
    {
        "query":    "carry forward leave",
        "positive": "Unused leave balance carries forward yearly.",
        "negative": "Refund process for orders."
    }
]
```

Each triplet teaches the model: 'For this query, this document is relevant (positive), and this one is not (negative).' The model is trained to minimize the distance between the query embedding and the positive document embedding, while maximizing the distance to the negative document. This is called contrastive learning.

Part B — Re-ranker Implementation (Cells 4–7)

```
docs = [
    "Shipping policy",
    "Refund for damaged laptops is 60 days",
    "Warranty details"
]

query = "damaged laptop refund"

def rerank(query, docs):
    """Score docs by counting overlapping query words — production
    systems use a trained cross-encoder instead."""
    q = set(query.split())
    return sorted(
        docs,
        key=lambda d: len(q & set(d.lower().split())),
        reverse=True
    )
```

```
print(rerank(query, docs))
# Output: ['Refund for damaged laptops is 60 days', ...]
```

The word-overlap reranker is a simple demonstration. In production, you would use a trained cross-encoder model that jointly encodes the query and each document together, producing a much more accurate relevance score. Libraries like sentence-transformers provide pre-trained cross-encoders you can fine-tune on your domain data.

Chapter 6: Vector Space Models and Embeddings in RAG

Section I — Theory & Conceptual Foundations

6.1 What Is a Vector Space Model?

A vector space model represents text as numerical vectors inside a high-dimensional mathematical space. Words, sentences, paragraphs, or documents become points in that space. Texts with similar meaning appear closer together; texts with different meaning appear farther apart. This mathematical structure is what enables semantic search — finding documents that mean the same thing as a query, even when they use different words.

6.2 Why Embeddings Transformed Retrieval

Traditional keyword search fails when a user asks 'How can I return a broken laptop?' but the document says 'Damaged laptops may be refunded within 60 days.' No shared keywords exist. Embeddings detect semantic similarity because they learn that broken ≈ damaged, return ≈ refunded, and laptop ≈ laptops from the statistical patterns in their training data.

6.3 Evolution of Text Representations

Representation Method	Key Characteristics
Bag of Words	Counts term frequencies — no semantic understanding, cannot handle synonyms
TF-IDF	Weights important words by inverse document frequency — still purely lexical
Word2Vec	First learned distributed word vectors — captures word-level semantic relationships
GloVe	Global co-occurrence vectors — improved training stability
FastText	Handles subword structure — better for morphologically rich languages
Transformer Embeddings	Contextual, state-of-the-art — the same word has different vectors in different contexts

6.4 Similarity Metrics

Metric	Description & Use Case
Cosine Similarity	Measures angle between vectors — most common for NLP. Score: A·B / (||A|| × ||B||)
Dot Product	Measures both direction and magnitude — common in retrieval systems

Euclidean Distance	Measures straight-line distance — sensitive to vector magnitude

6.5 Sparse vs Dense Vectors

Sparse (BM25/TF-IDF)	Dense (Embedding-Based)
Many dimensions, mostly zeros	Compact numeric vectors (hundreds to thousands of floats)
Keyword-focused — exact matches	Semantic-rich — captures meaning and synonyms
Very fast and interpretable	Requires embedding model inference
Best for exact term matching	Best for natural language queries

Section II — Practical & Coding

Parts A–F — Embedding Generation and Semantic Search

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from sklearn.metrics.pairwise import cosine_similarity

# Corpus
docs = [
    "Damaged laptops can be returned within 60 days.",
    "Annual leave balance carries forward yearly.",
    "Shipping takes 5 business days."
]

# Step 1: Generate embeddings for a single query
embedder = OpenAIEmbeddings()
vector   = embedder.embed_query("broken laptop refund")
print(f"Embedding dimensions: {len(vector)}")  # Typically 1536 for OpenAI

# Step 2: Store corpus in a vector database
db = Chroma.from_texts(docs, embedder)

# Step 3: Semantic search — different words, same meaning
results = db.similarity_search(
    "How do I return a broken notebook computer?",
    k=2
)
for r in results:
    print(r.page_content)

# Step 4: Manual cosine similarity demonstration
v1 = [[1, 2, 3]]
v2 = [[1, 2, 2]]
similarity = cosine_similarity(v1, v2)
print(f"Cosine similarity: {similarity[0][0]:.4f}")
```

- `embedder.embed_query("broken laptop refund")` — sends the text to the OpenAI embedding API and returns a dense vector of 1,536 floating-point numbers (for text-embedding-ada-002). This is the mathematical representation of the query's meaning.
- `Chroma.from_texts(docs, embedder)` — embeds each document in the corpus and stores the vectors in a local Chroma database. Each document is now a point in the 1,536-dimensional embedding space.
- `similarity_search("How do I return a broken notebook computer?", k=2)` — embeds the query, then finds the 2 nearest corpus vectors using cosine similarity. Returns 'Damaged laptops can be returned within 60 days.' — even though 'notebook computer' and 'broken' are different words from 'laptop' and 'damaged'.

Chapter 7: Expansion Strategies for RAG

Section I — Theory & Conceptual Foundations

7.1 Why Query Expansion Matters

One of the most common reasons RAG systems fail is poor user queries. Users ask vague, incomplete, shorthand, or typo-filled questions. A query like 'refund issue' could mean a damaged product refund, a delayed payment, a policy question, or a payment reversal. Expansion strategies transform weak queries into stronger retrieval inputs before the vector search runs.

7.2 Common Expansion Strategies

Strategy	Description & Example
Synonym Expansion	refund → return, reimbursement, restitution — adds semantically equivalent terms
Acronym Expansion	NLP → Natural Language Processing — prevents missed matches due to acronym mismatch
Entity Expansion	OpenAI → GPT, ChatGPT, API company — expands named entity variants
Domain Expansion	SLA → service level agreement — expands industry-specific abbreviations
Multilingual Expansion	loan policy → ऋण नीति — expands for multilingual corpora
Query Rewriting	Changes the query structure for clarity: 'leave carry' → 'Can unused annual leave balance carry forward?'

7.3 Advanced Techniques

Multi-Query Retrieval

Generate several alternate formulations of the original query and retrieve documents for each independently. Merge and deduplicate the result sets before re-ranking. This improves recall by covering different ways the relevant documents might be phrased.

HyDE — Hypothetical Document Embeddings

A powerful method: (1) Use the LLM to generate a hypothetical answer/document for the query, (2) Embed that generated text, (3) Retrieve real documents similar to the hypothetical one. This is particularly useful when the raw query is too short or abstract to produce good embeddings directly.

```
# HyDE concept
hypothetical_doc = """
Customers can return damaged laptops within 60 days
with original invoice proof, as per Returns Policy v4.
"""
```

```
# Embed this hypothetical answer
hypo_vector = embedder.embed_query(hypothetical_doc)

# Retrieve real documents similar to the hypothetical one
# db.similarity_search_by_vector(hypo_vector, k=5)
```

7.4 Common Risks

Risk	Description
Over-Expansion	Too many noisy synonyms flood the retrieval with irrelevant results
Intent Drift	The expanded query changes the user's original meaning — retrieves wrong domain
Higher Latency	Multiple retrieval calls (multi-query) add latency — needs caching or parallelization
Cost Increase	LLM-generated expansion (rewrites, HyDE) adds per-query LLM cost

Section II — Practical & Coding

Parts A–D — Expansion Pipeline

```
# Part A: Rule-based synonym expansion
synonyms = {
    "refund":  ["return", "reimbursement"],
    "laptop":  ["notebook", "computer"],
    "damaged": ["broken", "defective"]
}

query = "damaged laptop refund"

expanded_terms = []
for word in query.split():
    expanded_terms.append(word)
    expanded_terms.extend(synonyms.get(word, []))

expanded_query = " ".join(expanded_terms)
print(expanded_query)
# Output: damaged broken defective laptop notebook computer refund return
reimbursement

# Part B: Multi-query strategy
queries = [
    "damaged laptop refund",
    "broken notebook return policy",
    "defective computer reimbursement"
]
# Each query runs independently; merge results afterward

# Part C: LLM-based query rewriter
prompt = """
Rewrite this user query into 3 optimized retrieval queries:
```

```
damaged laptop refund
"""
# Send prompt to LLM; parse 3 reformulated queries from response
```

`synonyms.get(word, [])` — returns the synonym list for a word if it exists, or an empty list if not. This means words not in the synonym dictionary pass through unchanged — the original query intent is always preserved.

The multi-query strategy runs all three query variants and merges the result sets. Documents that appear in multiple result sets are natural candidates for higher confidence — they are relevant across multiple phrasings of the intent.

Chapter 8: Integrating Knowledge Bases for RAG Systems

Section I — Theory & Conceptual Foundations

8.1 Why Knowledge Integration Is the Hardest Part

A RAG system is only as valuable as the knowledge it can access. Many prototypes use a few PDFs. Production systems require integration with real enterprise knowledge sources: internal wikis, document repositories, databases, CRM systems, helpdesk tools, shared drives, APIs, and knowledge graphs. As one architect perspective puts it: knowledge integration is often the hardest part of enterprise AI — not the model itself.

8.2 Types of Knowledge Sources

Source Type	Examples	Integration Approach
Structured	SQL databases, ERP systems, CRM, inventory	JDBC queries, SQLAlchemy, ORM-based loaders
Semi-Structured	JSON APIs, logs, CSV files, HTML pages	REST API calls, pandas parsing, web scrapers
Unstructured	PDFs, Word docs, emails, meeting notes, policies	LangChain document loaders, OCR, pandoc

8.3 Integration Models

Model	Description & Best For
Batch Indexing	Periodically ingest all documents on a schedule (nightly, hourly). Best for static or slowly changing content like policy docs and manuals.
Real-Time Connectors	Fetch fresh data at query time via API calls. Best for rapidly changing content like inventory levels or current ticket status.
Hybrid Model	Static docs indexed + live APIs queried dynamically. The strongest enterprise pattern — combines freshness with search efficiency.

8.4 Core Design Challenges

Challenge	Why It Matters
Permissions	Users should only retrieve documents they are authorized to see — retrieval must be permission-aware
Freshness	Latest policy versions must be indexed; stale embeddings produce wrong answers confidently

Format Diversity	PDFs, tables, scanned images, and emails require different parsing strategies
Duplication	Same content in multiple systems creates conflicting answers and inflated index size
Metadata Quality	Without ownership, date, and department metadata, filtering and trust signals are weak

Section II — Practical & Coding

Parts A–E — Multi-Source Integration Pipeline

```
# Part A: PDF Integration
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("hr_policy.pdf")
docs   = loader.load()
# docs is a list of Document objects, one per PDF page

# Part B: SQL Database Integration
import sqlite3

conn   = sqlite3.connect("company.db")
cursor = conn.cursor()
cursor.execute("SELECT policy_text FROM policies")
rows = cursor.fetchall()
# Convert rows to text documents for embedding

# Part C: Live API Integration
import requests

data = requests.get("https://api.company.com/products").json()
# Extract text fields; embed for vector storage

# Part D: Convert to Vector Store with Metadata
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

texts = [
    "Refund policy is 60 days.",
    "Leave balance carries forward."
]
# Critical: attach metadata to every chunk
metadatas = [
    {"source": "hr_policy.pdf", "department": "HR", "updated_at": "2026-04-01"},
    {"source": "hr_policy.pdf", "department": "HR", "updated_at": "2026-04-01"},
]

db = Chroma.from_texts(texts, OpenAIEmbeddings(), metadatas=metadatas)

# Part E: Filtered Retrieval
results = db.similarity_search(
    "Can leave balance carry forward?",
    filter={"department": "HR"}  # Only search HR documents
)
```

- `PyPDFLoader("hr_policy.pdf").load()` — LangChain's PDF loader extracts text from each page, returning a list of Document objects with page content and metadata (page number, source filename). Each Document can be further split into smaller chunks before embedding.
- Metadata attached to each chunk enables filtered search — narrowing the search space to the correct department, year, or document type before vector comparison, dramatically improving both speed and precision.
- `filter={"department": "HR"}` — this metadata filter runs before the vector similarity search, restricting comparison to only HR documents. An HR query should never retrieve finance-only documents, even if their content is superficially similar.

Chapter 9: Relevance and Scoring Mechanisms for RAG

Section I — Theory & Conceptual Foundations

9.1 Why Scoring Determines Everything

In RAG, retrieval is not enough. A system may fetch many candidate documents — but which ones should be trusted and sent to the LLM first? That decision depends on relevance and scoring mechanisms. If the best chunk ranks #9, the LLM may never see it. Scoring directly impacts accuracy, faithfulness, latency (fewer chunks needed at higher quality), token cost, and user trust.

9.2 Major Scoring Types

Scoring Type	Mechanism	Best For
Lexical Scoring (BM25)	Term frequency × inverse doc frequency × length norm	Exact terms: policy numbers, product IDs, names
Semantic Scoring	Cosine similarity between query and document embeddings	Natural language queries, synonyms, paraphrases
Learned Re-ranking	Cross-encoder scores query-document pairs jointly	High-stakes top-K precision requirements
Hybrid Scoring	Weighted combination of BM25 + semantic scores	Most production systems — strongest overall quality

9.3 Hybrid Scoring Formula

```
# Combining BM25 and semantic similarity scores
bm25_score     = 0.70  # BM25 relevance (0-1 normalized)
semantic_score = 0.90  # Cosine similarity score (0-1)

# Weighted combination — tune weights on your golden dataset
final_score = 0.4 * bm25_score + 0.6 * semantic_score
print(f"Final score: {final_score:.3f}")
# Output: 0.820

# Often stronger than either alone because they capture
# complementary signals: exact-match AND semantic meaning
```

9.4 Multi-Stage Ranking Architecture

```
User Query
  ↓
Stage 1: Fast Retriever — retrieve top 100 candidates
         (ANN vector search — millisecond latency)
  ↓
Stage 2: Re-ranker — score top 20 with cross-encoder
         (more expensive but much more accurate)
  ↓
Stage 3: Final Context Selector — top 5 chunks
```

```
          (trim to fit LLM context window budget)
   ↓
LLM (receives only the 5 highest-quality chunks)
```

This three-stage architecture balances speed and quality: Stage 1 is fast but approximate, Stage 2 is slower but precise, Stage 3 manages token budget. Each stage narrows the candidate set, so later stages never pay the cost of re-scoring hundreds of documents.

9.5 Confidence Scoring and Thresholding

Modern systems estimate confidence by combining retrieval relevance, source authority, source freshness, and whether multiple sources agree. When confidence falls below a threshold, the system can: ask a follow-up clarification question, escalate to a human agent, or respond with an explicit 'I am not confident in this answer' caveat. This is far safer than generating a confident-sounding hallucination.

Section II — Practical & Coding

```
from rank_bm25 import BM25Okapi
from sklearn.metrics.pairwise import cosine_similarity

# ── BM25 Scoring ──────────────────────────────────────
docs   = [
    "Damaged laptops may be returned within 60 days.",
    "Shipping policy for electronics.",
    "Employee leave policy."
]
tokens = [d.lower().split() for d in docs]
bm25   = BM25Okapi(tokens)

query  = "damaged laptop refund"
bm25_scores = bm25.get_scores(query.lower().split())
print("BM25 scores:", bm25_scores)

# ── Semantic Similarity ───────────────────────────────
query_vec = [[1, 2, 3]]   # Replace with real embedding vector
doc_vec   = [[1, 2, 2]]
sem_score = cosine_similarity(query_vec, doc_vec)[0][0]
print(f"Semantic similarity: {sem_score:.4f}")

# ── Hybrid Final Score ────────────────────────────────
final_score = 0.4 * bm25_scores[0] + 0.6 * sem_score
print(f"Hybrid score: {final_score:.4f}")

# ── Confidence Score ──────────────────────────────────
relevance  = 0.92
authority  = 0.88
consensus  = 0.90
confidence = (relevance + authority + consensus) / 3
print(f"Confidence: {confidence:.3f}")

# Threshold check
if confidence < 0.70:
    print("Low confidence — escalate or ask for clarification")
```

Chapter 10: Multi-Modal RAG Systems

Section I — Theory & Conceptual Foundations

10.1 Why Multi-Modal RAG Is the Next Frontier

Traditional RAG works with text only. But real-world enterprise knowledge is not only text — it also exists in images, PDFs with charts, audio recordings, videos, tables, screenshots, diagrams, and scanned forms. Multi-modal RAG combines multi-modal data ingestion, modality-specific embeddings, cross-modal retrieval, and LLM/VLM generation to answer questions using evidence from any of these sources.

10.2 Multi-Modal vs Text RAG

Text RAG	Multi-Modal RAG
Only textual evidence	Combines images, audio, video, and text
Simpler, lower-cost pipeline	More complex — requires modality-specific parsers
Limited enterprise coverage	Unlocks high-value use cases: X-rays, charts, diagrams, invoices
Mature tooling	Rapidly evolving — CLIP, GPT-4V, LLaVA enable powerful capabilities

10.3 Common Multi-Modal Retrieval Patterns

Pattern	How It Works
OCR + Text Retrieval	Convert scanned documents/images to text with OCR, then apply standard text embedding and search
Native Image Embeddings	Use CLIP or similar to embed images as vectors, enabling visual similarity search
Video Transcript Retrieval	Transcribe spoken content, embed text, search by semantic similarity to spoken knowledge
Table-Aware Retrieval	Parse tables into structured rows, embed, and retrieve specific metrics or comparisons

Section II — Practical & Coding

Part A — OCR Pipeline for Scanned Documents (Cells 1–3)

```
pip install pytesseract pillow

from PIL import Image
```

```
import pytesseract

# Extract text from a scanned invoice image
img  = Image.open("invoice.png")
text = pytesseract.image_to_string(img)
print(text)
```

`pytesseract.image_to_string(img)` — runs Tesseract OCR on the image and returns extracted text as a string. Once you have the text, it can be chunked, embedded, and stored in the vector database exactly like any other text document. The OCR step is the bridge between visual content and the text-based RAG pipeline.

Parts B–D — Store OCR Text and Video Transcripts

```
# Store OCR-extracted text in vector database
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

db = Chroma.from_texts([text], OpenAIEmbeddings())
results = db.similarity_search("What is the invoice total?")

# Video transcript retrieval
transcript = """
Quarter four revenue grew by 18 percent year over year.
The growth was driven primarily by enterprise subscriptions.
"""
db_video = Chroma.from_texts([transcript], OpenAIEmbeddings())
results  = db_video.similarity_search("How much did Q4 revenue grow?")
print(results[0].page_content)
```

The same vector retrieval pattern applies to OCR text and transcripts as to standard documents. The modality-specific preprocessing (OCR, transcription) converts non-text content into text, after which the pipeline is identical. This is a clean architectural boundary — the retrieval layer is modality-agnostic once the input has been converted to text.

Chapter 11: Fact Verification with RAG Systems

Section I — Theory & Conceptual Foundations

11.1 Why Retrieval Alone Does Not Guarantee Truth

RAG reduces hallucinations by grounding answers in external knowledge, but retrieval alone does not guarantee truth. Retrieved documents may be outdated, contradictory, low quality, misleading, or incomplete. Fact verification asks: 'Can the model prove the answer?' — not just 'Can the model answer?'

11.2 Core Verification Pipeline

```
User Query
  ↓
Retrieve Evidence (multi-source)
  ↓
Rank Sources by Authority and Freshness
  ↓
Generate Draft Answer
  ↓
Verify: Does the answer match the evidence?
  ↓
Return Verified Answer + Citations  OR  Warning Flag
```

11.3 Types of Verification

Verification Type	Description
Source Grounding	Does every claim in the answer appear in the retrieved source documents?
Contradiction Detection	Do multiple retrieved sources give conflicting information on the same fact?
Freshness Verification	Is the source document recent enough to be authoritative for this query?
Multi-Source Consensus	Do multiple independent, reliable sources agree on the answer?
Numerical Validation	Are any numeric calculations in the answer mathematically consistent with the sources?

11.4 Confidence-Based Response Strategy

Confidence Level	System Behavior
High Confidence	Direct answer with citations — 'Based on Returns Policy v4 (March 2026): ...'

Medium Confidence	Answer with explicit caution note — 'Based on available information, which may not be fully current...'
Low Confidence	Ask clarifying question or escalate — 'I cannot find a confident answer. Would you like me to search further?'

Section II — Practical & Coding

```
# Source Grounding Check
answer = "Damaged laptops may be returned within 60 days."
source = "Official policy: Damaged laptops may be returned within 60 days."

verified = "60 days" in answer and "60 days" in source
print(f"Grounded: {verified}")

# Contradiction Detection
source_a = "Refund allowed within 60 days."
source_b = "Refund allowed within 30 days."

if source_a != source_b:
    print("Contradiction detected — sources conflict on refund period")

# Freshness Check
source_date     = "2026-03-01"
latest_required = "2026-01-01"  # Policy must be from at least Jan 2026

is_fresh = source_date >= latest_required
print(f"Source is fresh: {is_fresh}")

# Confidence Score
relevance  = 0.92
authority  = 0.88
consensus  = 0.90
confidence = (relevance + authority + consensus) / 3
print(f"Answer confidence: {confidence:.3f}")

if confidence >= 0.85:
    print("High confidence — return answer with citations")
elif confidence >= 0.70:
    print("Medium confidence — return with caution note")
else:
    print("Low confidence — escalate or ask for clarification")
```

Chapter 12: Federated Learning and Privacy-Preserving RAG

Section I — Theory & Conceptual Foundations

12.1 The Core Privacy Challenge

Many enterprises cannot simply upload internal documents to external AI services. Banks need regulatory compliance, hospitals protect patient records, governments require sovereign hosting, and enterprises protect IP and trade secrets. Privacy-preserving RAG systems are designed so that retrieval and generation happen while minimizing data exposure.

12.2 What Is Federated Learning?

Federated learning is an approach where data stays in local environments while model improvements are aggregated centrally. Instead of moving raw data to one server, each local site trains locally and shares only model weight updates — not the underlying data. For RAG, this means each data silo can be searched locally and only ranked evidence summaries are returned centrally.

12.3 Privacy Risks in Standard RAG

Risk	Description
Prompt Leakage	Sensitive data sent to external LLM APIs appears in logs or training data
Unauthorized Retrieval	Users access documents they should not see due to missing access controls
Logging Exposure	PII stored in query logs creates compliance liability
Embedding Leakage	Vector representations may encode sensitive content that can be partially reconstructed
Multi-Tenant Risks	One customer's data mixes with another's in shared vector index namespaces

12.4 Core Privacy Design Layers

Layer	Controls
Data Layer	Encrypt documents at rest; field-level encryption for PII fields
Retrieval Layer	Permission-aware search — users only retrieve authorized documents
Model Layer	Use private or self-hosted models for sensitive queries; external APIs only for safe content

Network Layer	Private VPC, zero-trust routing — no data traverses public internet
Governance Layer	Audit logs, access approvals, right-to-be-forgotten deletion pipelines

Section II — Practical & Coding

```
# Part A: Role-Based Document Filtering
documents = [
    {"text": "HR leave policy",        "role": "employee"},
    {"text": "Executive payroll file",  "role": "admin"},
    {"text": "Benefits overview",       "role": "employee"},
]

user_role = "employee"
allowed   = [d for d in documents if d["role"] == user_role]
print(f"Allowed documents: {len(allowed)}")  # Only 2 of 3

# Part B: PII Redaction Before Logging
import re

text = "Customer SSN 123-45-6789 requested refund."
redacted = re.sub(r"\d{3}-\d{2}-\d{4}", "***-**-****", text)
print(f"Safe to log: {redacted}")

# Part C: Secrets via Environment Variables
import os
api_key = os.getenv("OPENAI_API_KEY")  # Never hardcode

# Part D: Route Sensitive Queries to Private Model
query = "summarize executive payroll policy"

sensitive_keywords = ["payroll", "salary", "ssn", "medical"]
if any(kw in query.lower() for kw in sensitive_keywords):
    route = "private_model"   # Self-hosted LLM on private network
else:
    route = "external_model"  # External API (OpenAI, Anthropic, etc.)

print(f"Routing to: {route}")
```

- `d["role"] == user_role` — filters the document list to only those the current user's role is permitted to see. In a real system, this filtering happens in the vector database query itself (using metadata filters) rather than in Python, so that unauthorized documents are never even retrieved.
- `re.sub(r"\d{3}-\d{2}-\d{4}", "***-**-****", text)` — uses a regular expression to replace Social Security Number patterns with a redacted placeholder before the text is stored in logs. This is a basic PII redaction example — production systems use dedicated PII detection libraries.
- The sensitive keyword routing pattern ensures that queries about payroll, medical records, or other sensitive topics never reach external LLM APIs — they are always routed to a private, self-hosted model where data cannot leave the enterprise network.

Chapter 13: Cross-Lingual RAG Systems

Section I — Theory & Conceptual Foundations

13.1 Why Multilingual RAG Matters

Most organizations operate across languages, regions, and cultures. An English-only RAG system becomes limiting when users ask questions in Hindi, Punjabi, Tamil, Spanish, French, Arabic, or mixed-language text. A Cross-Lingual RAG system allows users to ask questions in one language while retrieving knowledge stored in another — opening AI access beyond language barriers.

13.2 Types of Multilingual RAG

Type	How It Works
Monolingual + Translation	Translate the user query to the corpus language first, then run standard retrieval
Multilingual Embedding Retrieval	Use multilingual embedding models that represent multiple languages in a shared vector space
Cross-Lingual Generation	Retrieve evidence in one language; generate the answer in the user's language
Mixed-Language (Hinglish/Spanglish)	Handle code-switching where users mix languages within a single query

13.3 Why Multilingual Embeddings Are Key

Multilingual embedding models (like multilingual-e5 or mBART) place semantically similar text from different languages near each other in vector space. This means 'refund policy', 'रिफंड नीति', and 'politique de remboursement' can all be close to each other — enabling cross-lingual retrieval without explicit translation of every document.

13.4 India-Specific Opportunities

Use Case	Multilingual Capability Needed
Banking Bots	Hindi + English + regional languages — EMI, GST, Aadhaar terminology
Government Schemes	Citizen access to scheme information in local language (22 scheduled languages)
Education	Bilingual STEM explanations — English concepts explained in Hindi
E-commerce	Customer support in vernacular: Marathi, Telugu, Kannada, Bengali

Section II — Practical & Coding

```
# Part A: Language Detection
from langdetect import detect

query = "लैपटॉप रिफंड पॉलिसी क्या है?"
lang  = detect(query)
print(f"Detected language: {lang}")  # Output: 'hi' (Hindi)

# Part B: Translation Flow (Conceptual)
# In production, use Google Translate API or a local translation model
translated_query = "What is the laptop refund policy?"
# Then run standard retrieval on the English corpus

# Part C: Multilingual Embedding Retrieval
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

docs = ["Damaged laptops may be returned within 60 days."]
db   = Chroma.from_texts(docs, OpenAIEmbeddings())

# Query in Hindi — multilingual embeddings handle cross-lingual retrieval
results = db.similarity_search("लैपटॉप रिफंड नीति")
print(results[0].page_content)

# Part D: Answer in User's Language
prompt = """
Use the retrieved context below to answer the question.
Answer in Hindi.

Context: Damaged laptops may be returned within 60 days.
Question: लैपटॉप रिफंड पॉलिसी क्या है?
"""
# LLM response: 'लैपटॉप 60 दिनों के भीतर लौटाए जा सकते हैं।'

# Part E: Hinglish (Mixed-Language) Query Handling
query = "Laptop refund policy kya hai?"
# Detect as mixed; try retrieval in both English and Hindi indices
# Merge results; rank; generate answer
```

- `detect(query)` — the langdetect library identifies the language of a text string and returns a language code (e.g., 'hi' for Hindi, 'en' for English, 'fr' for French). This is the first step in a multilingual RAG pipeline — you cannot handle a multilingual query correctly without first knowing what language it is in.
- The prompt explicitly instructs the LLM to answer in Hindi, regardless of which language the retrieved documents are in. Modern LLMs like GPT-4 and Claude handle cross-lingual generation natively — they can read English evidence and produce fluent Hindi, Bengali, or Tamil responses.
- Hinglish handling is a practical challenge for Indian enterprise systems. A robust approach: detect mixed-language, expand to both English and Hindi variants, retrieve from both indices, and merge results before ranking.

Chapter Summary — Key Takeaways Across All 13 Chapters

Production & Architecture

Core Deployment Principles

1. Production RAG requires two distinct pipelines: offline ingestion and online serving.
2. Containerize each component (API, ingestion worker, retriever) independently with Docker.
3. Use Kubernetes for auto-scaling, rolling updates, load balancing, and failover at scale.
4. Schedule data refresh jobs to prevent stale knowledge from reaching users.
5. Adopt modular RAG architecture — every component should be independently swappable.
6. Monitor latency, retrieval quality, cost, and hallucination rates continuously.

Retrieval Quality

Retrieval Engineering Rules

7. Hybrid retrieval (BM25 + vector) almost always outperforms either alone.
8. Multi-stage ranking: fast retriever (100 candidates) → re-ranker (20) → context selector (5).
9. Metadata filtering reduces search scope before vector comparison — faster and more precise.
10. Optimize chunk size (≈800 tokens) with overlap (≈100 tokens) to prevent boundary failures.
11. Query expansion strategies (synonyms, multi-query, HyDE) improve recall for weak queries.
12. Fine-tune embedding models only when clear retrieval pain is measured — not based on hype.

Evaluation & Trust

Measurement and Verification

13. Measure retrieval (Recall@K, MRR, NDCG) and generation (Faithfulness, Relevance, Correctness) separately.
14. Build a golden dataset (benchmark queries) and run it on every release as a regression suite.
15. Fact verification — grounding checks, contradiction detection, freshness validation — is a system layer.
16. Confidence scoring should drive escalation and human-in-the-loop review for low-confidence outputs.

Security & Privacy

Privacy-First Design

17. Never send sensitive documents to external LLM APIs by default — route based on content sensitivity.
18. Role-based retrieval filtering: users should only see documents they are authorized to access.

19. Redact PII before logging; encrypt vector stores at rest; use namespace isolation for multi-tenant systems.
20. Federated RAG enables enterprise knowledge access without centralizing sensitive raw data.

Advanced Capabilities

Next-Generation RAG Features

21. Multi-modal RAG extends retrieval to images (OCR), video (transcripts), and tables.
22. Cross-lingual RAG uses multilingual embeddings to retrieve across languages without translating every document.
23. Hinglish and mixed-language queries require detection, dual-index retrieval, and merged ranking.
24. For AI architects, retrieval quality is often the real competitive advantage — not model size.